CCNA 200-301 V2.0

31 Ansible and Configuration Management

Part V — Wireless, Virtualisation and Operations

EXAM TOPICS

5.5

LECTURER

Dr. Mustafa Hameed

THE ISLAMIA UNIVERSITY OF BAHAWALPUR

Department of Information Technology

What you will be able to do

- **Explain** what agentless means and why it matters for network devices.
- **Read** an inventory file and a playbook, and say what each line does.
- **Use** Ansible to run show commands across many devices and to push a configuration change.
- **Recognise** the three failures that account for most broken playbooks: YAML indentation, the wrong module, and a change that was never saved.

Before we start

Chapter 23 for SSH and credentials — Ansible logs in exactly as you do. Chapter 30 for the five management approaches; this chapter is the automation-based one made concrete. Chapter 7 for the commands being pushed.

KEY TERMS

Vocabulary for this chapter

- agentless
- idempotent
- control node
- managed node
- inventory
- play
- task
- module
- playbook
- facts
- YAML
- dry run
- vault

Each is defined where it first matters — not here.

ON THE EXAM

The verb is use

This is what the blueprint actually asks for

Topic **5.5** is *use configuration management mechanisms such as Ansible to execute commands*. It is the only topic in domain 5 that is not *describe*, *select* or *interpret* — so you are expected to be able to read a playbook and say what it will do to which devices.

"Such as Ansible" leaves the door open to Puppet, Chef and SaltStack, but every published Cisco example uses Ansible, and Ansible is the one whose agentless model actually suits network hardware. Learn Ansible; know the other names and the one-line difference.

DEFINITION

Configuration management

Describing the state a set of machines should be in, and using a tool to make them match that description — repeatably, and with a record of what was done.

WATCH OUT

This is the whole reason Ansible dominates network automation

A server can run an agent. A Catalyst switch cannot — it is an appliance, and you may not install arbitrary software on it. Any tool that requires an agent is therefore unavailable for the devices this exam is about.

Ansible needs nothing on the device except the SSH access you already configured in Chapter 23. That is the answer to “why Ansible for networks”, and it is a fair exam question.

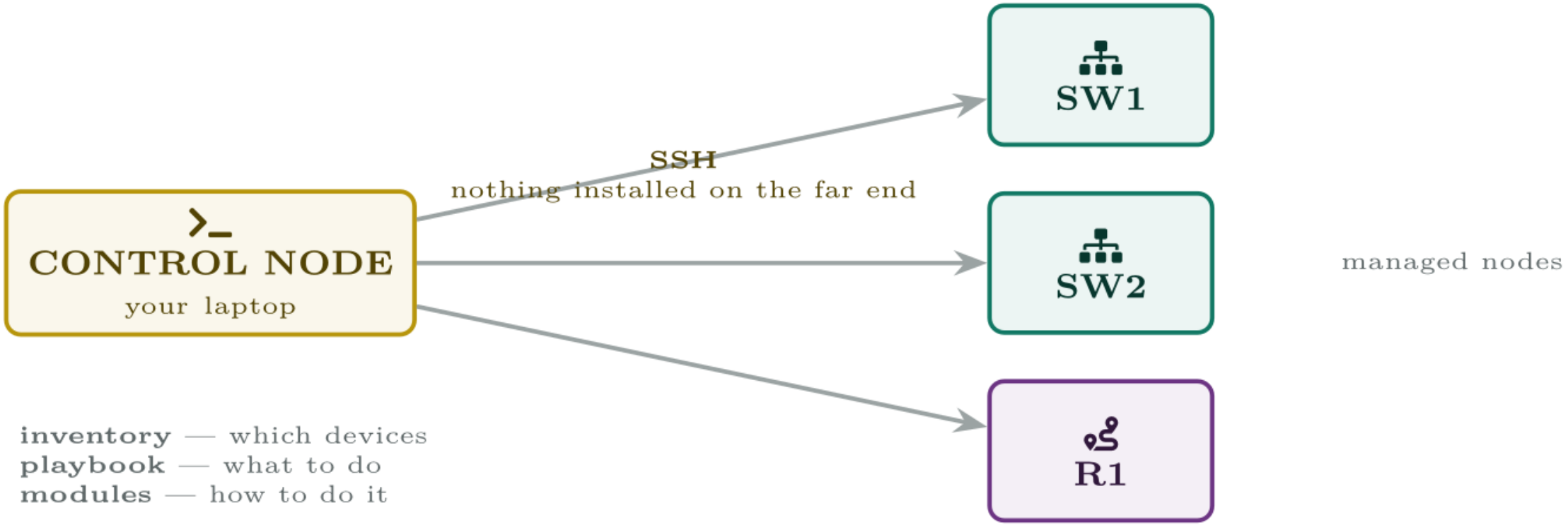
DEFINITION

Idempotent

An operation that produces the same result whether it is applied once or twenty times. Ansible tasks are written to describe a desired state, so a task that has already been satisfied reports ok and changes nothing.

This is what makes it safe to run the same playbook every day. A script that blindly types commands is not idempotent; a playbook that says “this line should be present” is.

The pieces



Everything lives on the control node. The devices need only the SSH access they already have.

Inventory.yml

Ansible · YAML

```
all:
  children:
    access_switches:
      hosts:
        SW1: { ansible_host: 10.0.0.11 }
        SW2: { ansible_host: 10.0.0.12 }
        SW3: { ansible_host: 10.0.0.13 }
    routers:
      hosts:
        R1: { ansible_host: 10.0.0.1 }
  vars:
    # network_cli is essential. Without it Ansible assumes the far end
    # is a Linux host with a Python interpreter, and every task fails
    # with an error that does not mention the real problem.
    ansible_connection: ansible.netcommon.network_cli
    ansible_network_os: cisco.ios.ios
    ansible_user: netadmin
```

WATCH OUT

Never put the password in the inventory

Use `ansible-vault` to encrypt a variables file, and supply the key at run time:

Then run with `–ask-vault-pass`, or use SSH keys and no password at all. A plain-text credential in a file that is about to be committed to version control is how credentials leak, and “it is only the lab” is how the habit forms.

VERIFY

ansible access_switches -i inventory.yml -m cisco.ios.ios_command -a "commands='show version'"

Linux or macOS

```
SW1 | SUCCESS => {
    "changed": false,
    "stdout_lines": [["Cisco IOS XE Software, Version 17.12.04", "..."]]
}
SW2 | SUCCESS => {
    "changed": false,
    "stdout_lines": [["Cisco IOS XE Software, Version 17.09.05", "..."]]
}
```

CONFIGURE

Audit.yml — collect and report

Ansible · YAML

```
---
- name: Audit access switches
  hosts: access_switches
  gather_facts: false

  tasks:
    - name: Collect version and inventory facts
      cisco.ios.ios_facts:
        gather_subset:
          - hardware
      register: device_facts

    - name: Show what we found
      debug:
        msg: "{{ inventory_hostname }} runs {{ ansible_net_version }}"

    - name: Run a show command and keep the output
      cisco.ios.ios_command:
        commands:
          - show interfaces status
      register: intf
```

Audit.yml — collect and report (continued)

Ansible · YAML

```
    - name: Write it to a file on the control node
      copy:
        content: "{{ intf.stdout[0] }}"
        dest: "./reports/{{ inventory_hostname }}-interfaces.txt"
      delegate_to: localhost
```

Vlan.yml — make a change on every switch

Ansible · YAML

```
---
- name: Add the guest VLAN everywhere
  hosts: access_switches
  gather_facts: false

  tasks:
    - name: Create VLAN 40
      cisco.ios.ios_config:
        lines:
          - name GUEST
        parents: vlan 40

    - name: Allow it on the uplink trunk
      cisco.ios.ios_config:
        lines:
          - switchport trunk allowed vlan add 40
        parents: interface GigabitEthernet1/0/48

    # Without this the change lives in the running configuration only
    # and is lost at the next reload. "modified" saves only when this
    # run actually changed something.
```

Vlan.yml — make a change on every switch (continued)

Ansible · YAML

```
    - name: Save the configuration
      cisco.ios.ios_config:
        save_when: modified
```

WATCH OUT

ios_command reads, ios_config writes

This is the distinction the exam will test and the one beginners get wrong.

`ios_command` runs commands in EXEC mode and returns their output. It **cannot make a configuration change**, and it will refuse configuration commands outright rather than silently failing.

`ios_config` enters configuration mode and applies lines, using `parents` to say which sub-mode they belong in. It compares against the running configuration first, so it is idempotent.

A playbook that reports success and changes nothing is very often `ios_command` where `ios_config` was meant.

ansible-playbook -i inventory.yml vlan.yml –check –diff

Linux or macOS

```
TASK [Create VLAN 40] *********************************************
changed: [SW1]
changed: [SW2]
ok: [SW3]

PLAY RECAP ********************************************************
SW1  : ok=2  changed=1  unreachable=0  failed=0
SW2  : ok=2  changed=1  unreachable=0  failed=0
SW3  : ok=3  changed=0  unreachable=0  failed=0
```

CONCEPT

Read the recap, every time

`--check` is a dry run: Ansible reports what it *would* do and touches nothing. `--diff` shows the exact lines. Run this before every change to production, without exception.

In that output, `SW3` reports `ok` rather than `changed` because it already has VLAN 40. That is idempotency visible in the recap, and it is also a useful audit: the difference between the two tells you which devices had drifted.

`unreachable` is a different failure from `failed`: unreachable means Ansible could not log in at all, failed means it logged in and the task did not work.

Common pitfalls

- **Tabs in YAML.** A syntax error, and the message rarely says “tab”. Configure your editor to insert spaces.
- **Missing `ansible_connection: network_cli`.** Ansible tries to run Python on a switch and fails confusingly.
- **`ios_command` where `ios_config` was meant.** Reports success, changes nothing.
- **No `save_when`.** The change works until the device reloads.
- **Wrong `parents`.** Lines are applied in global configuration mode instead of under the interface, which either errors or does something you did not intend.
- **Skipping `--check`.** There is no undo across forty devices.

Common pitfalls (continued)

- **Credentials in the inventory file.** Use vault or SSH keys.
- **Running against `all` when you meant one group.** Check the `hosts:` line before every run. It is the single most expensive typo available to you.

The playbook reports success on all forty switches. After a power cut, eleven have lost the change.

A new VLAN was added across the access layer with a playbook. Every device reported changed, spot checks confirmed the VLAN existed, and the change was signed off. Three weeks later a power event reboots part of the estate, and eleven switches come back without VLAN 40. The twenty-nine that did not lose power still have it.

The playbook reports success on all forty switches. After a power cut, eleven have lost the change.

What would you check first — and what do you expect to see?

show archive config differences nvram:startup-config system:running-config

```
SW1#
!Contextual Config Diffs:
+vlan 40
+ name GUEST
```

What is the fault — and what does this output rule out?

Why it is doing that

Diagnosis. There is no `save_when` task. `ios_config` writes to the *running* configuration, exactly as typing the commands would — and like typing them, it does not save. Every switch was correctly configured and none of them wrote that configuration to `startup-config`.

The evidence pattern confirms it precisely: the split is not between devices that the playbook succeeded on and devices it failed on. It is between devices that **rebooted** and devices that did not. Twenty-nine switches still have the VLAN because they have been up the whole time and are still running the configuration Ansible applied. The eleven that power-cycled reloaded `startup-config`, which never contained it.

Why it is doing that

Any fault that correlates with reboots rather than with the change itself points at the running-versus-startup distinction.

Vlan.yml — as it was run

Ansible · YAML

```
---
- name: Add the guest VLAN everywhere
  hosts: access_switches
  gather_facts: false

  tasks:
    - name: Create VLAN 40
      cisco.ios.ios_config:
        lines:
          - name GUEST
        parents: vlan 40
```

Vlan.yml — corrected

Ansible · YAML

```
    - name: Save the configuration
      cisco.ios.ios_config:
        save_when: modified
```

And the lesson

Fix. Add the save task and re-run.

`save_when: modified` writes only when the run actually changed something, so re-running against the twenty-nine devices that are already correct costs nothing. The eleven get the VLAN and a save.

The verification lesson. The spot check confirmed the VLAN existed — in the running configuration. It should have compared running against startup:

Any output at all from that command means unsaved changes are present. Making it the last step of every change — by hand or as a final Ansible task — closes this class of fault permanently.

Automating a change you would otherwise do forty times

Platform note. Ansible needs real SSH and a real Python environment on the control node. Packet Tracer cannot do this. Use CML or GNS3 with three or more IOS devices and a Linux VM, or install Ansible on your own machine and point it at the lab.

1. Install Ansible and the `cisco.ios` collection on the control node. Record the versions of both.
2. Configure SSH and a local user on three devices as in Chapter 23. Confirm you can log in by hand before involving Ansible at all — if this does not work, nothing after it will.
3. Write `inventory.yml` with two groups. Verify with `ansible-inventory -list`.
4. Run an ad-hoc `ios_command` for `show version` against one group. Then deliberately remove `ansible_connection` and run it again. Record the error and explain why it does not mention the real cause.

Automating a change you would otherwise do forty times (continued)

1. Store the password with `ansible-vault` and run using `–ask-vault-pass`. Confirm the file is unreadable without the key.
2. Write `audit.yml` to gather facts and write a per-device report file on the control node. Confirm every task reports `changed: false` and explain why.
3. Write `vlan.yml` to add a VLAN and allow it on a trunk. Run with `–check –diff` first. Record the diff, then run for real.
4. Run it a second time with no changes. Confirm every device reports `ok` rather than `changed`. This is idempotency; write down what you would conclude if one device reported `changed` on the second run.

Automating a change you would otherwise do forty times (continued)

1. **Break 1.** Reproduce the Break & Fix. Omit `save_when`, apply a change, reload a device, and confirm the loss. Then add the save task and repeat.
2. **Break 2.** Replace `ios_config` with `ios_command` for the VLAN task. Record exactly what Ansible reports and what the device shows.
3. **Break 3.** Introduce a tab character into the playbook. Record the error message and how long it takes you to find it.
4. **Extension.** Add a final task that runs the `show archive config differences` command from this chapter and fails the play if it returns anything. You now cannot leave a change unsaved.

CHECKPOINT 1 of 6

What does agentless mean, and why does it decide the choice of tool for network devices?

Discuss · then advance

ANSWER

Checkpoint 1 of 6

What does agentless mean, and why does it decide the choice of tool for network devices?

> It installs nothing on the managed device and connects over SSH. It decides the choice because you cannot install an agent on a Catalyst switch — so any agent-based tool (Puppet, Chef) is simply unavailable for network hardware, while Ansible needs only the SSH access already configured.

CHECKPOINT 2 of 6

What is idempotency, and where do you see it in a play recap?

Discuss · then advance

ANSWER

Checkpoint 2 of 6

What is idempotency, and where do you see it in a play recap?

> An operation produces the same result whether applied once or many times. In the recap you see it as `ok` rather than `changed` on a device that is already in the desired state — so a second run of the same playbook reports `changed=0`.

CHECKPOINT 3 of 6

State the difference between `ios_command` and `ios_config`.

Discuss · then advance

ANSWER

Checkpoint 3 of 6

State the difference between `ios_command` and `ios_config`.

> `ios_command` runs EXEC commands and returns output; it cannot make a configuration change. `ios_config` enters configuration mode and applies lines. A playbook reporting success while changing nothing is very often the former where the latter was meant.

CHECKPOINT 4 of 6

What does `save_when: modified` do, and what fails without it?

Discuss · then advance

ANSWER

Checkpoint 4 of 6

What does `save_when: modified` do, and what fails without it?

> It writes the running configuration to startup-config, but only when that run actually changed something. Without it the change lives in the running configuration only and is lost at the next reload — which shows up weeks later as a fault correlating with reboots rather than with the change.

CHECKPOINT 5 of 6

What is the difference between `unreachable` and `failed` in the recap?

Discuss · then advance

ANSWER

Checkpoint 5 of 6

What is the difference between `unreachable` and `failed` in the recap?

> `unreachable` means Ansible could not log in to the device at all. `failed` means it logged in and the task did not work. They point at completely different problems.

CHECKPOINT 6 of 6

What do `--check` and `--diff` do, and when would you omit them?

Discuss · then advance

ANSWER

Checkpoint 6 of 6

What do `-check` and `-diff` do, and when would you omit them?

> `-check` is a dry run — it reports what it would do and touches nothing. `-diff` shows the exact lines. Omit them only when you have already run with them and are applying the change you just reviewed.

What to take away

- Ansible is **agentless**: it connects over SSH and installs nothing. That is why it works on switches, where Puppet and Chef cannot.
- Tasks are **idempotent** — they describe desired state, so running twice changes nothing the second time.
- Playbooks are YAML: spaces not tabs, `-` for list items, `key: value` with a space after the colon.
- The inventory names devices and groups, and must set `ansible_connection: network_cli` and `ansible_network_os`.
- `ios_command` reads; `ios_config` writes, with `parents` to select the sub-mode.

What to take away (continued)

- `save_when: modified` writes to startup-config. Without it the change survives only until the next reload.
- `--check --diff` is a dry run. Use it before every production change.
- In the recap, `changed` means it acted, `ok` means it was already correct, `unreachable` means it never logged in.

Where we got to

- Ansible is **agentless**: it connects over SSH and installs nothing. That is why it works on switches, where Puppet and Chef cannot.
- Tasks are **idempotent** — they describe desired state, so running twice changes nothing the second time.
- Playbooks are YAML: spaces not tabs, `-` for list items, `key: value` with a space after the colon.
- The inventory names devices and groups, and must set `ansible_connection: network_cli` and `ansible_network_os`.

NEXT

Chapter 32 — AI in Network Operations